

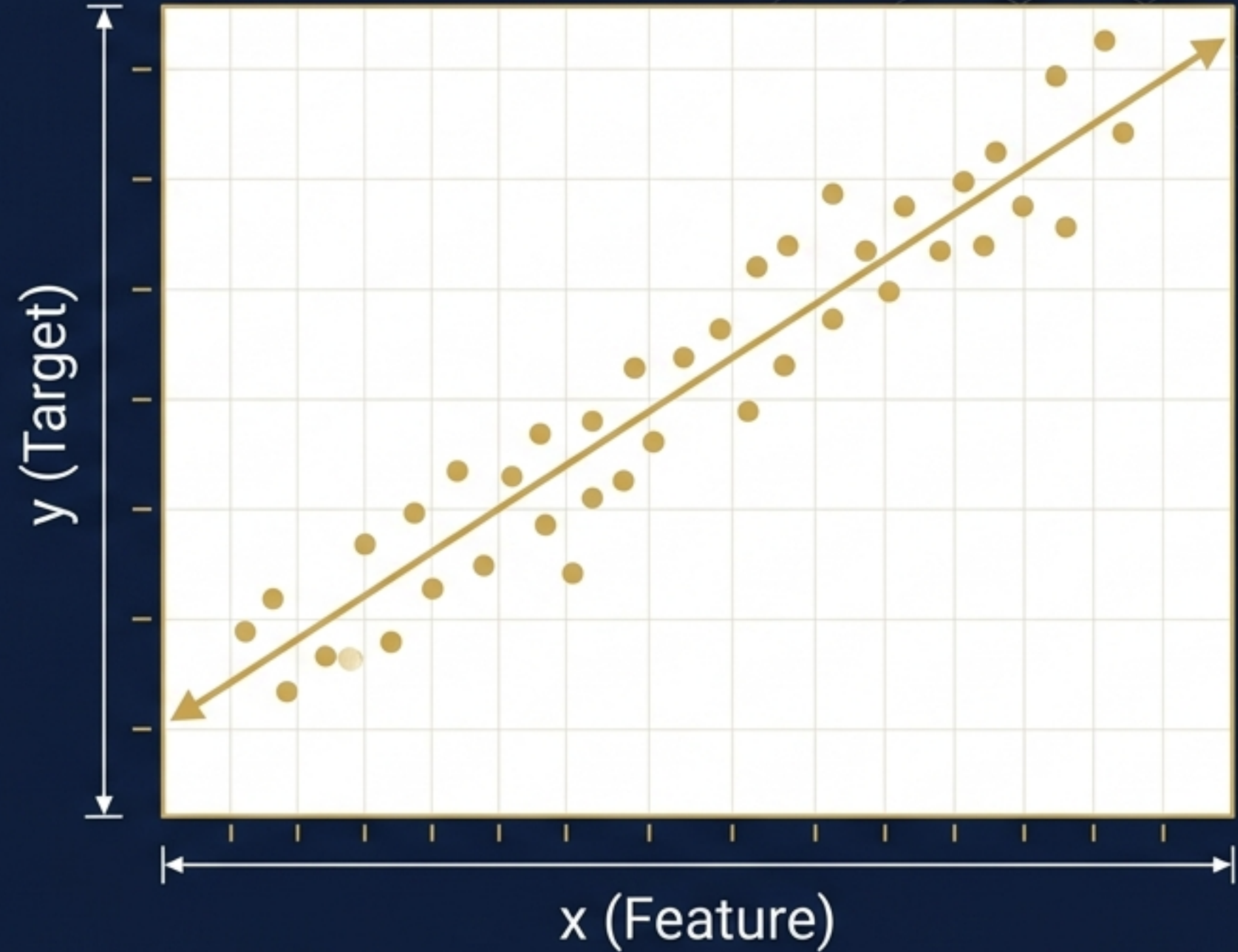
NEURAL NETWORKS

Parsa Salamatipour



LINEAR REGRESSION RECAP

$$y = (\underbrace{w}_{\substack{\text{Weight} \\ \text{(Slope)}}} \times x) + \underbrace{b}_{\substack{\text{Bias} \\ \text{(Intercept)}}$$



THE ARTIFICIAL PERCEPTRON



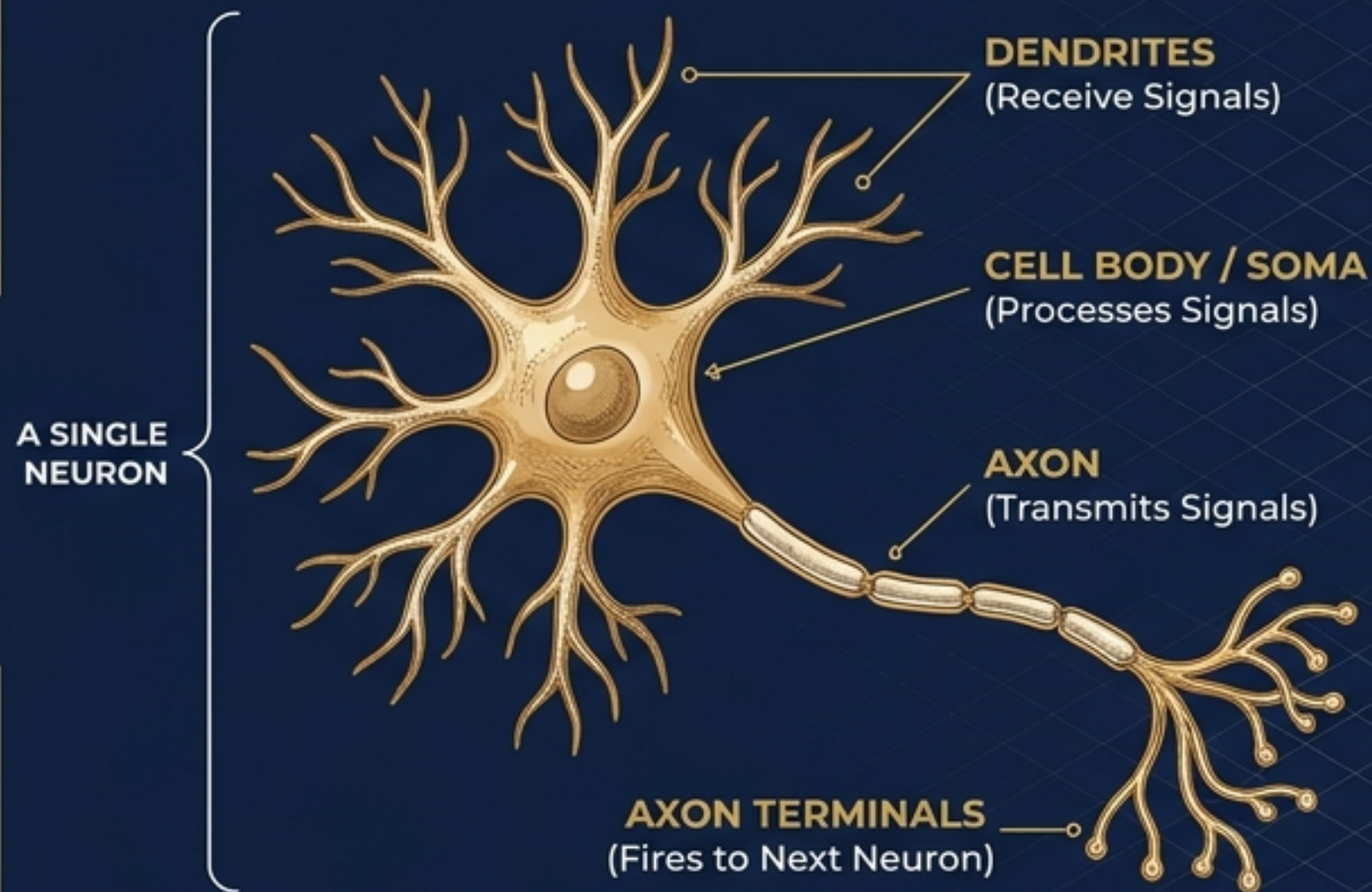
$$z = (x_1 \times w_1) + (x_2 \times w_2) + (x_3 \times w_3) + b$$

$$(2 \times 0.5) + (3 \times -1) + (4 \times 2) + 1 = 7$$

A single perceptron is a multi-dimensional linear regression

THE BIOLOGICAL INSPIRATION

Why a neuron?



We steal this idea: Multiple inputs, a processing unit, and a single output.

THE PERCEPTRON GRAVEYARD

One Layer vs. Deep Stack

SHALLOW (1 HIDDEN LAYER)



DEEP (2 HIDDEN LAYERS)



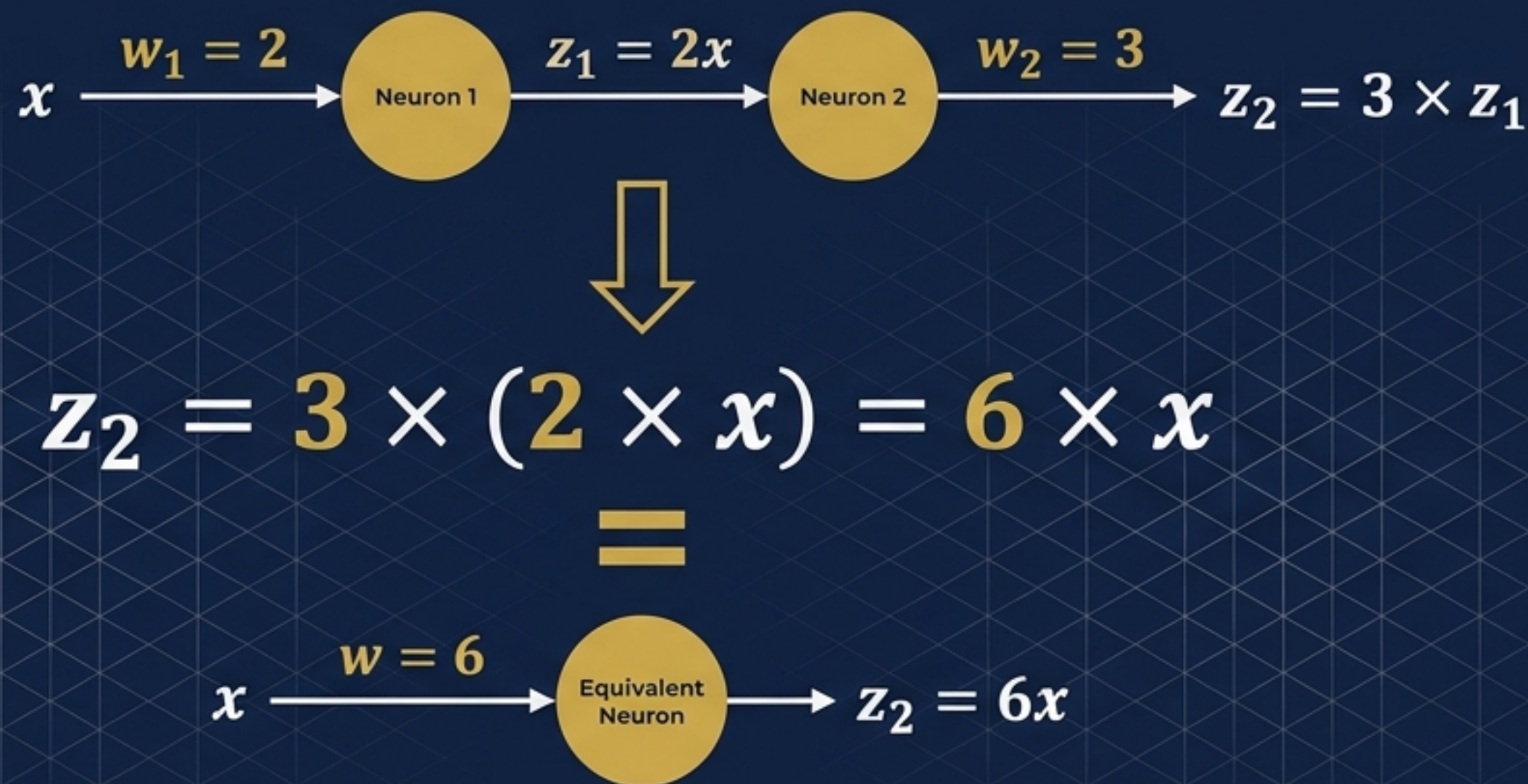
?

What happens mathematically when
you stack linear regressions?

Hint: If Neuron A outputs '2x' and Neuron B multiplies that by '3'... what is the final result?

THE LINEAR STACKING FAILURE

The Setup for ReLU



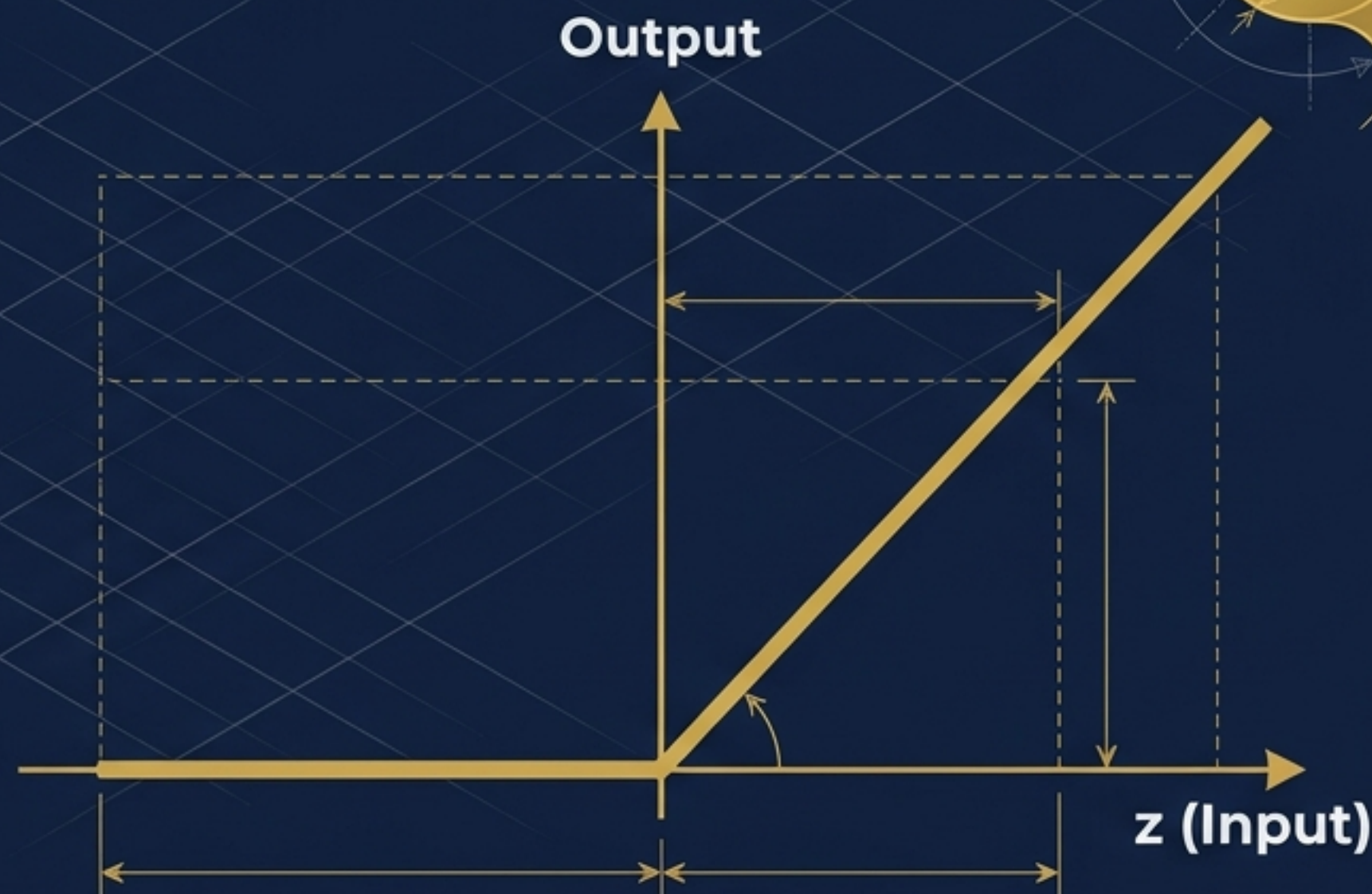
Stacking linear layers without activation functions is mathematically equivalent to a single linear layer. A network with 100 layers is just a straight line.

We need a non-linear "wrench" to break this chain...

THE WRENCH: ACTIVATION FUNCTION

Introducing ReLU

If $z \leq 0 \rightarrow \text{Output} = 0$
If $z > 0 \rightarrow \text{Output} = z$

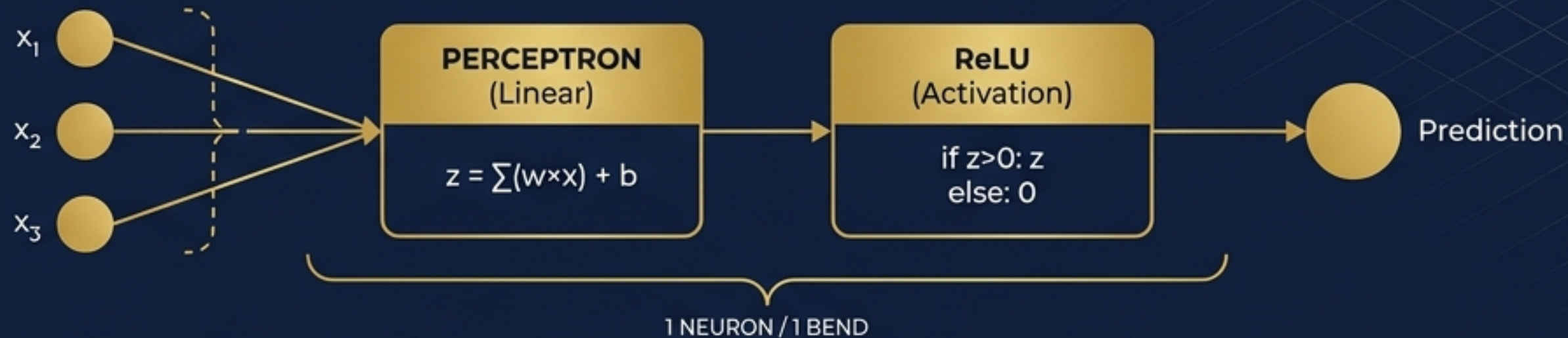


ReLU breaks the linear chain. A straight line that folds at zero creates a 'bend'.
Stack enough bends and you can model ANY curve in the universe.

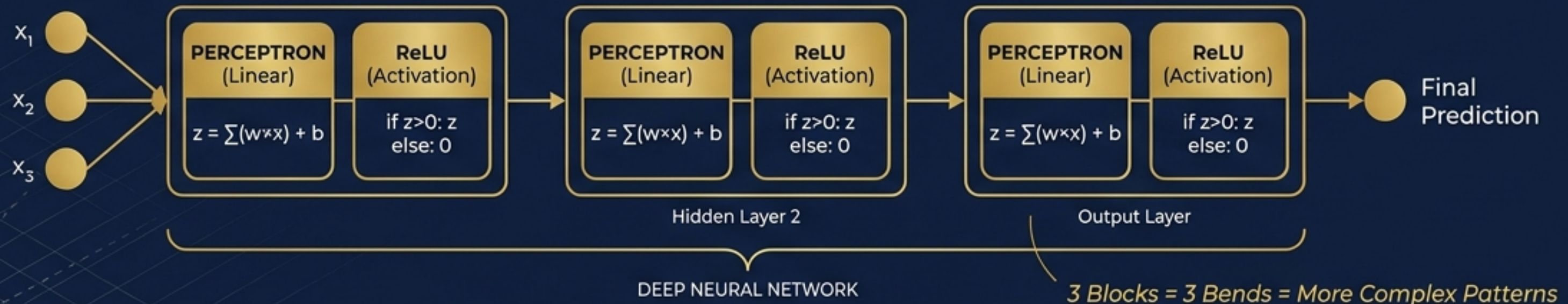
BUILDING THE TRUE NEURAL NETWORK

The Block Diagram

THE BUILDING BLOCK



STACKING THE BLOCKS



**The Perceptron (Linear) + ReLU (Non-Linear) is our fundamental building block.
Stack them to create powerful models.**

HOW IT PREDICTS: FORWARD PROPAGATION

The One-Way Trip



One complete forward pass produces one prediction.

Forward Propagation = `model.predict()`. Data enters from the left, flows forward through the layers, and exits as a single number on the right. **No learning** happens here yet—just pure computation.

THE PUNISHMENT: MEAN SQUARED ERROR

How Wrong Are We?

We need to penalize this gap.

Prediction:
\$240,000

Actual:
\$300,000

$$\begin{aligned}\text{Error} &= \text{Prediction} - \text{Actual} \\ &= \$240,000 - \$300,000 \\ &= -\$60,000\end{aligned}$$

Error₁ = -60k

Error₂ = +20k

Error₃ = -10k

$$\text{MSE} = \frac{(\text{Error}_1^2 + \text{Error}_2^2 + \dots + \text{Error}_n^2)}{n}$$

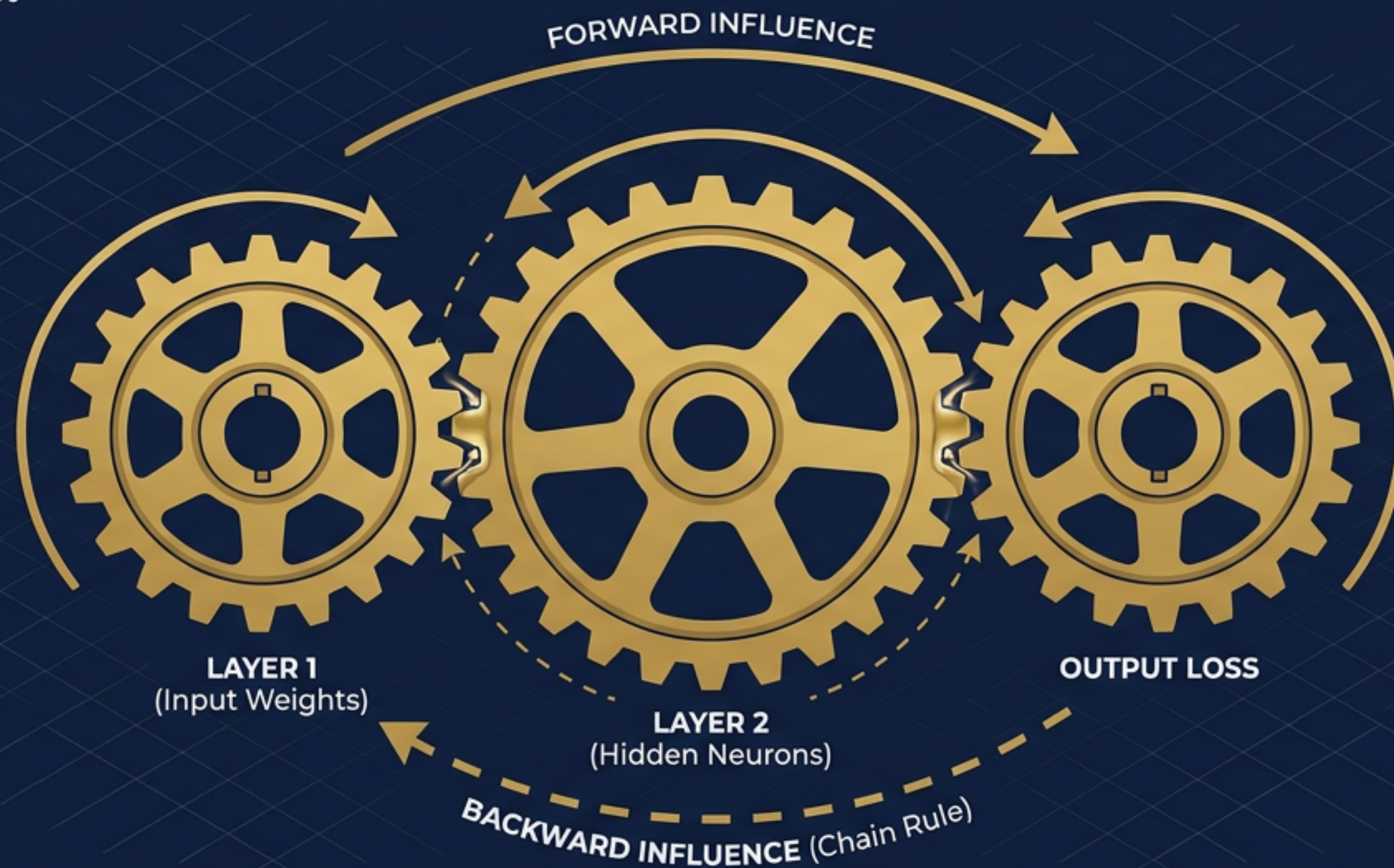
Square each error (makes them all positive) →
Sum them → Divide by the number of samples.
This gives us **ONE** number: the **LOSS**.

Loss = 0 means perfect prediction. Our network's only goal is to minimize this number.

Loss tells us exactly how "punished" the network should be.
A high loss = a big punishment. A low loss = a small punishment.
This single number drives everything the network learns.

THE CHAIN OF INFLUENCES

The Gears Analogy



If Gear 3 (Loss) changes by 1 degree, how much does Gear 1 (Weights) change?

We must trace the chain of influences backward:

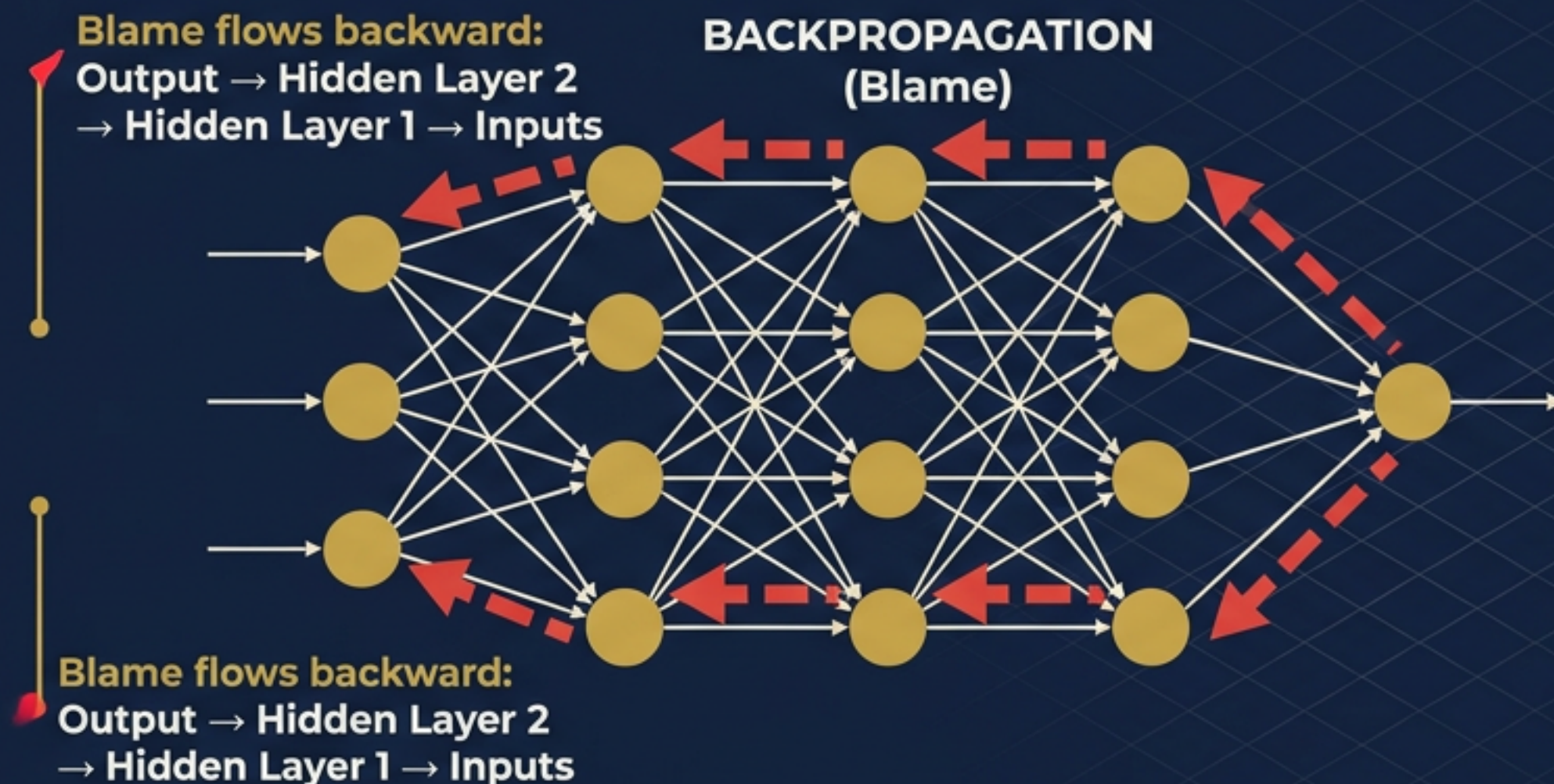
Gear 3 ← Gear 2 ← Gear 1.

Next slide: How this applies to our neural network.

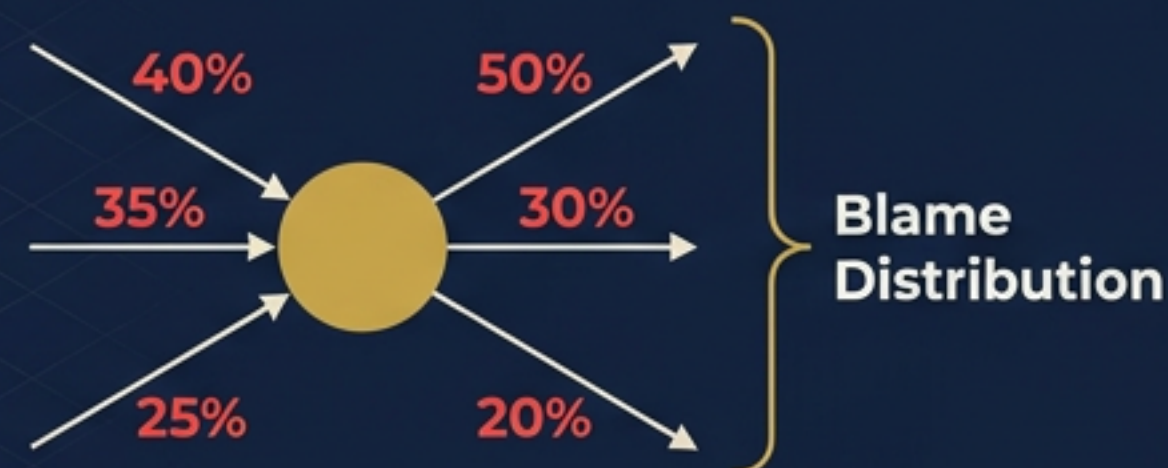
The **network** is a **mechanical system**. To adjust the first layer, we must understand how its rotation affects every subsequent layer all the way to the final loss. This "**chain of influences**" is the foundation of **Backpropagation**.

VISUALIZING BACKPROPAGATION

The Blame Game



Each neuron asks: 'How much of the final error am I responsible for?'

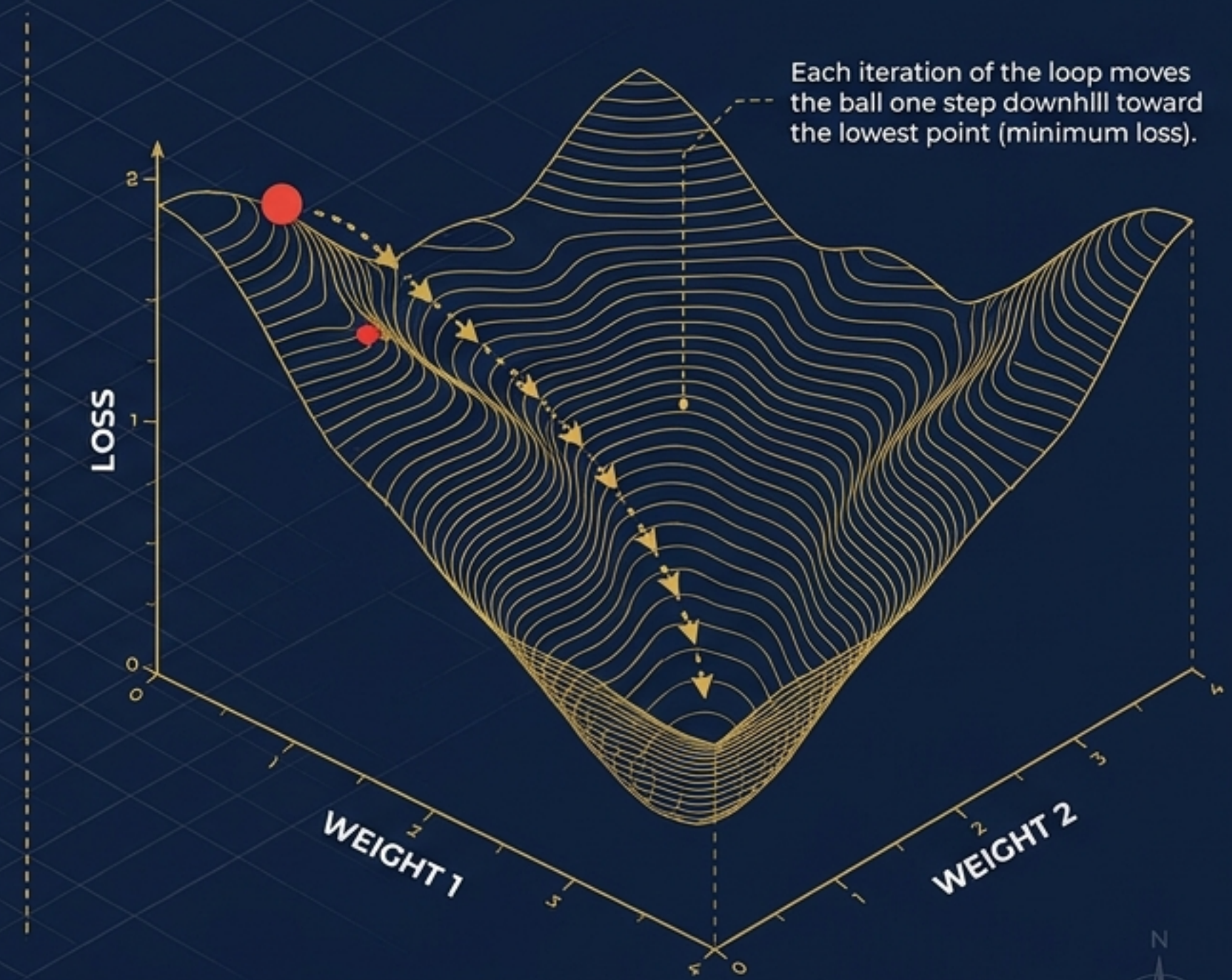


Larger weights → More blame
→ Larger adjustments.

Backpropagation calculates the 'blame' for every single weight in the network. The weight with the largest blame gets the largest adjustment. This is how the network learns which connections actually matter.

THE GRAND TRAINING LOOP

The Cycle of Learning



Training = Repeating this cycle thousands of times.
Forward → Measure → Backpropagate → Update.
Each cycle nudges the ball closer to the valley floor.

The number of cycles = Epochs.

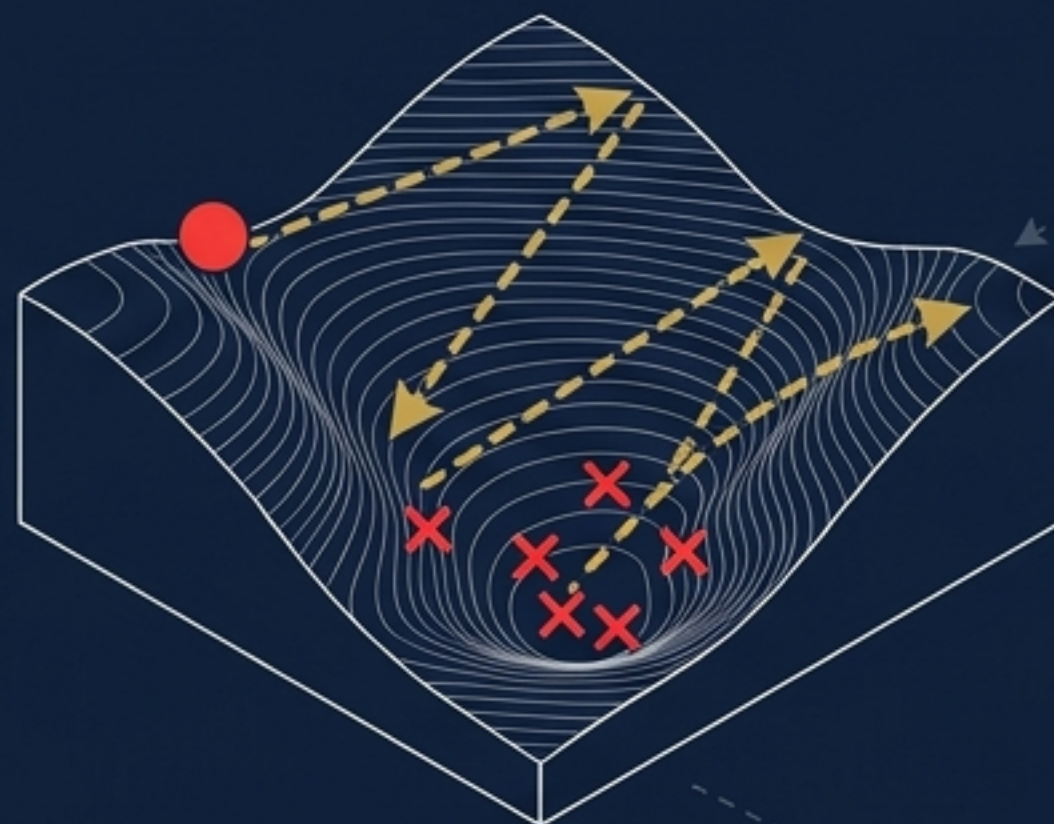
THE STEP SIZE: LEARNING RATE

How Big Should Our Steps Be?



LEFT LANDSCAPE

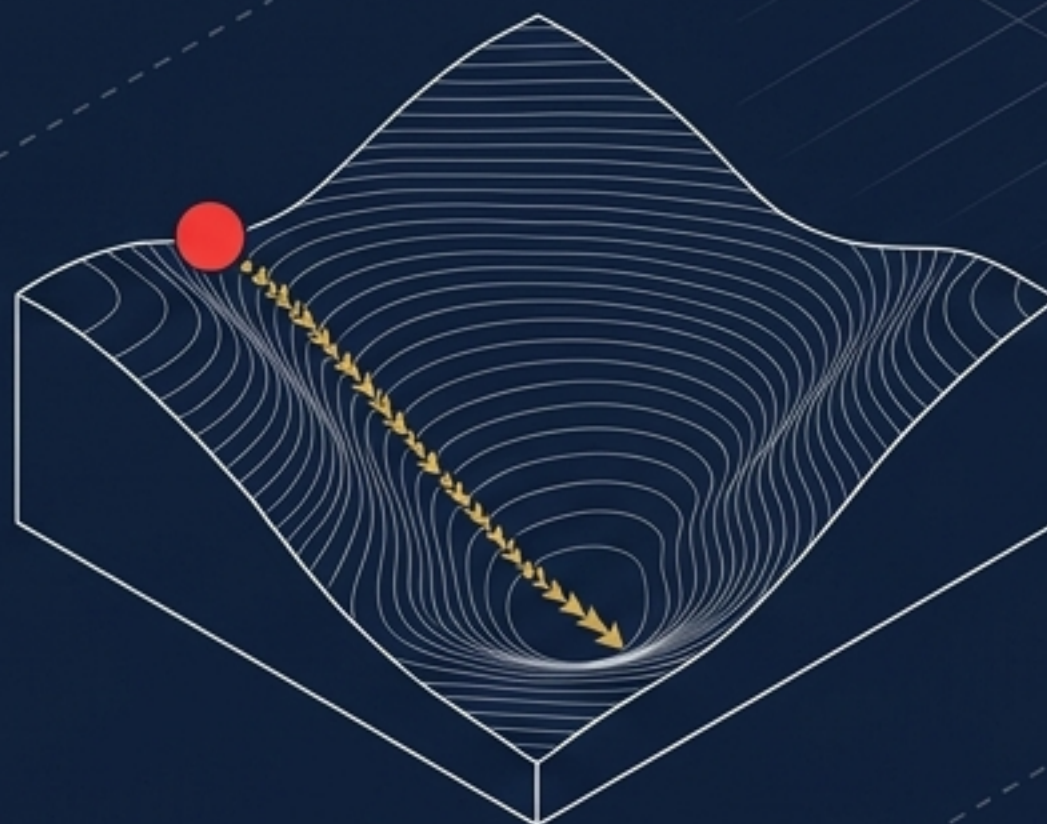
TOO BIG



Learning Rate Too High → Overshoots the valley. Loss bounces wildly and never converges.

CENTER LANDSCAPE

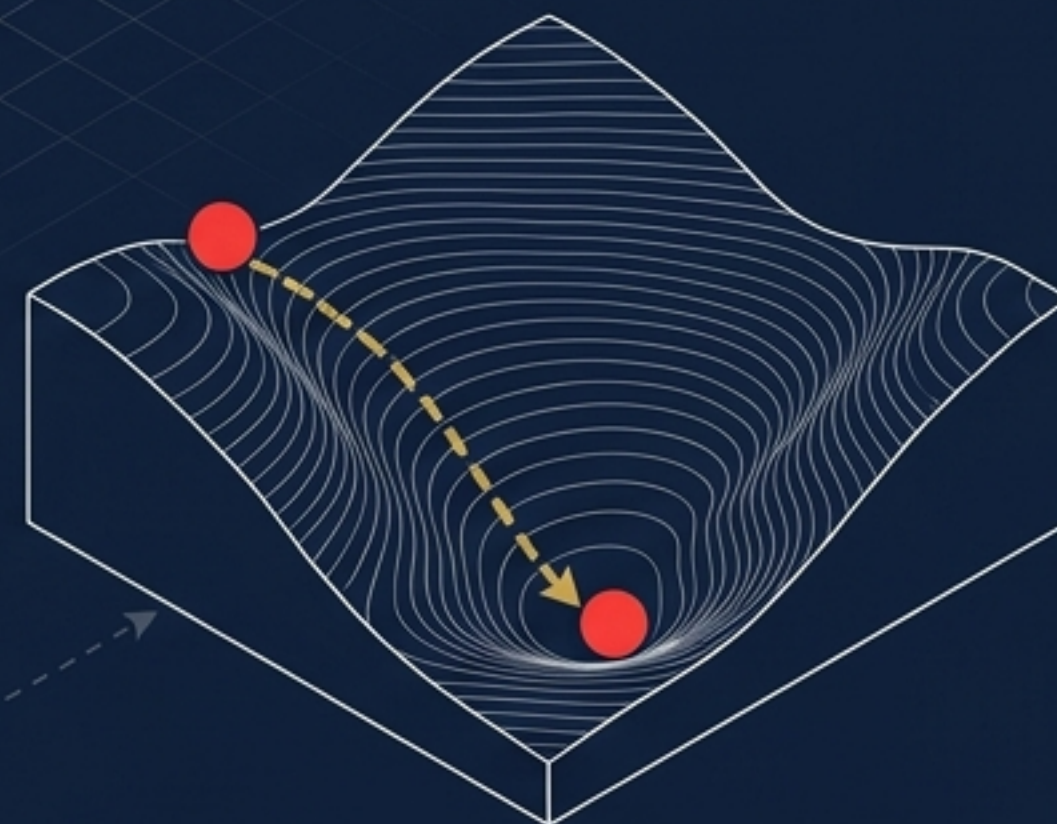
TOO SMALL



Learning Rate Too Small → Crawls down the slope. Takes forever to reach the bottom. Wastes training time.

RIGHT LANDSCAPE

JUST RIGHT



Learning Rate Just Right → Smooth, confident steps straight to the minimum. Optimal convergence.

**The Learning Rate is the most important dial you will turn.
Too big = chaos. Too small = wasted time. Just right = efficient learning.**

Finding the sweet spot is a skill you develop with practice.

THE SUPERPOWERS: USES AND EXCITEMENT

What Can Neural Networks Do?

UNSTRUCTURED DATA

Pixels, Audio, Text, Sensor Data



FACE RECOGNITION



SELF-DRIVING CARS



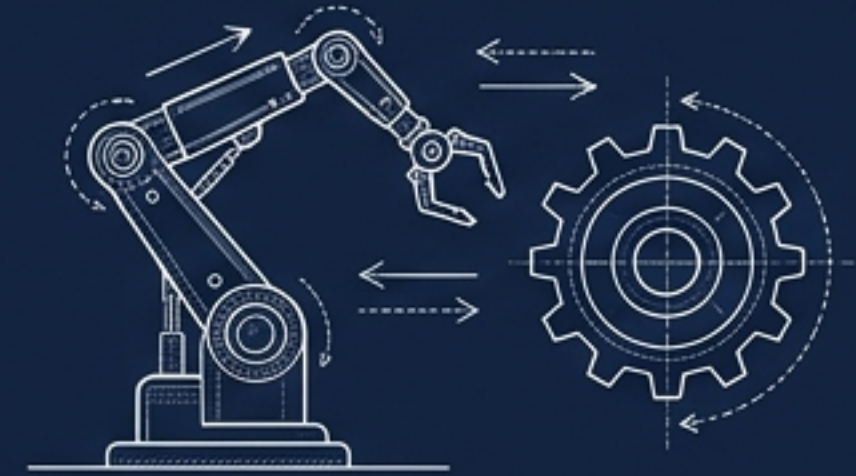
LANGUAGE & TRANSLATION



MEDICAL DIAGNOSIS



RECOMMENDATION SYSTEMS



ROBOTICS & CONTROL

Neural Networks automatically learn features from raw data. No manual feature engineering required. This is why they power modern AI—from ChatGPT to self-driving cars.

What took us 15 slides to understand took decades of research to discover. You now know the engine that runs the AI revolution.

